Unidad 02

Principios Avanzados de Programación Orientada a Objetos

Biblioteca de Plantillas Estándar (STL)

Índice

Introducción 4
Contenedores 5
a) Contenedores de secuencia 5
b) Contenedores asociativos 5
c) Contenedores no ordenados 5
d) Adaptadores de contenedores 6
Iteradores 6
Algoritmos 7
Ejemplo práctico: 8
Ejemplo Integrador (To-Do List sencilla en C++) 9
Errores Comunes de Programación 10
Buenas Prácticas de Programación 11

Introducción

La STL (Standard Template Library) es un conjunto de clases y funciones genéricas que forman parte de la biblioteca estándar de C++.

Su objetivo es reutilizar estructuras y algoritmos ya optimizados, reduciendo el tiempo de desarrollo y los errores comunes en la programación.

La STL se apoya fuertemente en el concepto de plantillas (templates), permitiendo trabajar con cualquier tipo de dato sin reescribir código.

Componentes principales:

1. **Contenedores**: estructuras de datos genéricas (vector, list, stack, etc.).
2. **Iteradores**: objetos que permiten recorrer los elementos de los contenedores.
3. **Algoritmos**: funciones que realizan operaciones comunes (búsqueda, ordenamiento, conteo, transformación…).
4. **Functores (function objects)**: objetos que se comportan como funciones y se pueden pasar como parámetros a los algoritmos.

Componentes	Rol principal	Ejemplo
Contenedores	Guardan los datos.	`std::vector`, `std::list`
Iteradores	Acceden secuencialmente a los datos sin conocer su estructura.	`begin()`, `end()`
Algoritmos	Operan sobre los datos usando iteradores.	`std::sort`, `std::find`

Esta separación permite que **los algoritmos sean independientes del tipo de contenedor**, lo que es una de las grandes ventajas del diseño genérico de C++.

Las **plantillas** son un mecanismo del lenguaje C++ que permite definir **funciones o clases genéricas**, capaces de trabajar con distintos tipos de datos sin duplicar código. Son la base de la STL, ya que todos sus componentes están definidos como **templates**.

Ejemplo de plantilla de función:

```
template <typename T>
T maximo(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    cout << maximo(10, 5) << endl;       // int
    cout << maximo(3.5, 7.2) << endl;    // double
}
```

Contenedores

Los **contenedores** almacenan colecciones de objetos. Se dividen en tres grandes grupos:

a) Contenedores de secuencia

Almacenan elementos en un orden lineal.

Contenedor	Descripción	Ejemplo
`std::array`	Tamaño fijo, acceso rápido y seguro.	`std::array<int,5> arr = {1,2,3,4,5};`
`std::vector`	Tamaño dinámico, acceso aleatorio eficiente.	`vec.push_back(6);`
`std::deque`	Inserción/eliminación en ambos extremos.	`dq.push_front(0);` `dq.push_back(4);`
`std::list`	Lista doblemente enlazada (acceso secuencial).	`lst.push_front(0);` `lst.push_back(5);`
`std::forward_list`	Lista enlazada simple (más liviana).	`fl.push_front(1);`

b) Contenedores asociativos

Almacenan elementos en pares **clave–valor** de manera ordenada (implementados como árboles balanceados).

Contenedor	Descripción
`std::set`	Conjunto de elementos únicos ordenados.
`std::map`	Pares clave–valor ordenados por la clave.
`std::multiset`	Permite duplicados.
`std::multimap`	Permite múltiples valores por clave.

c) Contenedores no ordenados

Utilizan tablas hash (rendimiento promedio O(1)).

Contenedor	Descripción
`std::unordered_set`	Conjunto sin orden definido.
`std::unordered_map`	Pares clave–valor sin orden definido.

d) Adaptadores de contenedores

Ofrecen una interfaz simplificada sobre otros contenedores.

Adaptador	Estructura	Descripción
`std::stack`	Pila (LIFO).	`stk.push(10); stk.top();`
`std::queue`	Cola (FIFO).	`q.push(1); q.front();`
`std::priority_queue`	Cola con prioridad (mayor primero).	`pq.push(20); pq.top();`

Iteradores

Un **iterador** es un objeto que apunta a elementos dentro de un contenedor. Permiten recorrer el contenido sin importar la estructura interna.

Tipos de iteradores:

- `begin()` → primer elemento
- `end()` → posición siguiente al último
- `rbegin()` / `rend()` → recorrido inverso
- `cbegin()` / `cend()` → iteradores constantes

Tipo	Descripción	Contenedores compatibles
Input Iterator	Solo lectura, recorrido hacia adelante.	`istream_iterator`
Output Iterator	Solo escritura, recorrido hacia adelante.	`ostream_iterator`
Forward Iterator	Lectura/escritura hacia adelante.	`forward_list`
Bidirectional Iterator	Permite avanzar y retroceder.	`list`, `set`, `map`
Random Access Iterator	Permite saltar posiciones arbitrarias.	`vector`, `deque`, `array`

Ejemplo:

```
std::vector<int> vec = {1, 2, 3, 4, 5};
for (auto it = vec.begin(); it != vec.end(); ++it)
    std::cout << *it << " ";
```

Algoritmos

Los **algoritmos STL** son funciones genéricas que operan sobre rangos de iteradores.

Principales categorías:

- **Búsqueda:** `std::find`, `std::count`, `std::binary_search`
- **Ordenamiento:** `std::sort`, `std::reverse`
- **Modificación:** `std::copy`, `std::remove_if`, `std::transform`
- **Acumulación y reducción:** `std::accumulate` (de `<numeric>`)
- **Aplicación de funciones:** `std::for_each`

Algoritmo	Función	Ejemplo
`std::sort()`	Ordena los elementos.	`sort(v.begin(), v.end());`
`std::reverse()`	Invierte el orden.	`reverse(v.begin(), v.end());`
`std::find()`	Busca un valor.	`find(v.begin(), v.end(), x);`
`std::count()`	Cuenta ocurrencias.	`count(v.begin(), v.end(), 3);`
`std::unique()`	Elimina duplicados consecutivos.	`unique(v.begin(), v.end());`
`std::remove_if()`	Elimina elementos que cumplan una condición.	`remove_if(v.begin(), v.end(), esPar);`
`std::transform()`	Aplica una función a cada elemento.	`transform(v.begin(), v.end(), v.begin(), cuadrado);`
`std::accumulate()`	Suma o acumula valores.	`accumulate(v.begin(), v.end(), 0);`

Ejemplo práctico:

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <numeric>
using namespace std;

int main() {
    vector<int> vec = {3, 1, 4, 1, 5};

    sort(vec.begin(), vec.end());          // Ordenar
    int suma = accumulate(vec.begin(), vec.end(), 0); // Sumar
    auto it = find(vec.begin(), vec.end(), 4);        // Buscar
    if (it != vec.end()) cout << "Encontrado: " << *it << endl;

       // Multiplica por 2
    for_each(vec.begin(), vec.end(), [](int& n){ n *= 2; });
    for (int n : vec) cout << n << " ";
}
```

Un **functor** (function object) es una clase que sobrecarga el operador `()`, permitiendo que sus objetos actúen como funciones.
Son útiles cuando se necesitan funciones personalizadas en algoritmos STL.

Ejemplo de functor:

```
struct Multiplicar {
    void operator()(int& n) const { n *= 2; }
};

for_each(vec.begin(), vec.end(), Multiplicar());
```

Hoy en día, los **lambda expressions** suelen reemplazarlos por su simplicidad:

```
for_each(vec.begin(), vec.end(), [](int& n){ n *= 2; });
```

Ejemplo Integrador (To-Do List sencilla en C++)

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<string> tareas;
    string tarea;
    int opcion;

    do {
        cout << "\n1. Agregar tarea\n2. Mostrar tareas\n3.
Eliminar tarea\n0. Salir\nOpcion: ";
        cin >> opcion; cin.ignore();

        if (opcion == 1) {
            cout << "Ingrese tarea: ";
            getline(cin, tarea);
            tareas.push_back(tarea);
        }
        else if (opcion == 2) {
            cout << "\nTareas:\n";
            for_each(tareas.begin(), tareas.end(), [](const
string& t){ cout << "- " << t << endl; });
        }
        else if (opcion == 3) {
            cout << "Ingrese tarea exacta a eliminar: ";
            getline(cin, tarea);
            tareas.erase(remove(tareas.begin(), tareas.end(),
tarea), tareas.end());
        }
    } while (opcion != 0);

    return 0;
}
```

Errores Comunes de Programación

- Usar índices fuera del rango (`vector[i]` sin validar tamaño).
- Invalidar iteradores al eliminar o insertar elementos.
- Mezclar tipos de contenedores sin plantillas adecuadas.
- No incluir los encabezados necesarios (`<vector>`, `<list>`, `<algorithm>`, etc.)
- No usar `#include <numeric>` para `accumulate`.
- Olvidar que `std::set` y `std::map` **mantienen los elementos ordenados automáticamente** y no permiten duplicados (a diferencia de `multiset` o `multimap`).

Buenas Prácticas de Programación

- **Usar `auto`** para evitar errores con tipos largos de iteradores.
- **Preferir `const` y referencias** para evitar copias innecesarias.
- **Inicializar contenedores** en el momento de la declaración.
- **Evitar modificar contenedores mientras se recorren** con iteradores.
- **Usar algoritmos STL antes que bucles manuales** (más eficientes y legibles).
- **Liberar memoria automáticamente**: los contenedores manejan su propia memoria, no uses `new` ni `delete` con ellos.